

## ADMIN-HANDBUCH

# Antrag 3000

Die Förder-Datenbank zentral pflegen – Admin-Werkzeug

---

Version 0.2.0 · Stand Juni 2026 · Eigenständige Admin-App (Desktop)

**Nur für die verwaltende Person.** Dieses Handbuch beschreibt die **Admin-App** – getrennt von der Nutzer-App. Sie pflegt die **unkritische** Förder-Datenbank (Katalog, Meldungen, geteilte Förderer). Sensible Tresor-Daten der Nutzer:innen sieht sie nie.

# Inhalt

|           |                                      |           |
|-----------|--------------------------------------|-----------|
| <b>1</b>  | Was die Admin-App ist                | <b>3</b>  |
| <b>2</b>  | Sicherheit: Zwei Faktoren            | <b>4</b>  |
| <b>3</b>  | Server vorbereiten (TOTP einrichten) | <b>5</b>  |
| <b>4</b>  | Admin-App installieren & starten     | <b>6</b>  |
| <b>5</b>  | Anmelden                             | <b>7</b>  |
| <b>6</b>  | Vorschläge kuratieren                | <b>8</b>  |
| <b>7</b>  | Meldungen bearbeiten                 | <b>9</b>  |
| <b>8</b>  | Geteilte Förderer                    | <b>10</b> |
| <b>9</b>  | Katalog hochladen                    | <b>11</b> |
| <b>10</b> | Der Sammler (Hybrid-Update)          | <b>13</b> |
| <b>11</b> | App-Updates veröffentlichen          | <b>14</b> |
| <b>12</b> | Tipps & häufige Fragen               | <b>15</b> |

# 1 Was die Admin-App ist

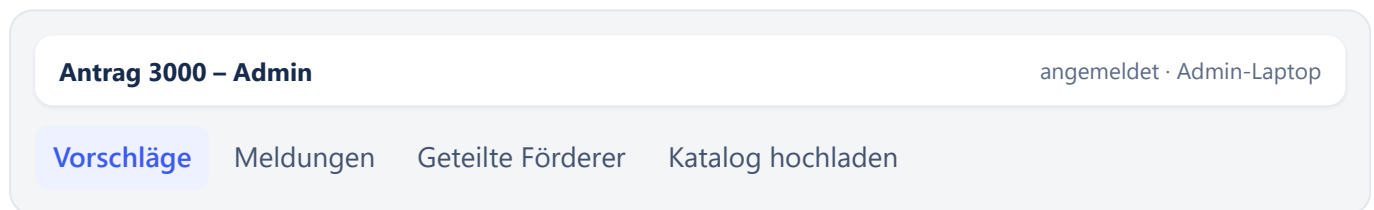
Ein schlankes, **eigenes** Werkzeug, mit dem die verwaltende Person die Förder-Datenbank des Teams zentral pflegt.

Die Admin-App ist **von der Nutzer-App getrennt** und hat einen eigenen Zweck: Sie verteilt und kuratiert die **unkritische** Förder-Ebene. Sie speichert selbst **keine** sensiblen Daten.

## Was du damit tust

- **Vorschläge** des Sammlers freigeben oder verwerfen,
- **Meldungen** der Nutzer:innen (falsche/veraltete Förderungen) bearbeiten,
- vom Team **geteilte Förderer** sichten und unpassende entfernen,
- den **Förder-Katalog hochladen** – er wird danach an alle Geräte verteilt.

**Zwei Datenebenen.** Wie die ganze App trennt auch die Admin-Seite strikt: Der Server (und damit die Admin-App) kennt nur die **öffentliche** Förder-Ebene. Name, IBAN, Budget, Projektbeschriebe der Nutzer:innen bleiben verschlüsselt auf deren Geräten.



**Abb. 1.1** – Nach dem Anmelden: vier Bereiche für die Pflege.

## 2 Sicherheit: Zwei Faktoren

Ein Admin kann zentral viel bewegen – darum ist der Zugang **zwei-faktorig** abgesichert.

### ① Besitz · Team-Gerät

Ein gültiges Team-Zertifikat (aus dem Zugangs-Paket).  
Caddy lässt nur solche Geräte überhaupt durch (mTLS).

### ② Wissen/Besitz · TOTP

Ein 6-stelliger Code aus deiner Authenticator-App, der alle 30 s wechselt.

**Abb. 2.1** – Beide Faktoren müssen stimmen, sonst kein Admin-Zugang.

Beim Anmelden prüft der Server den TOTP-Code und gibt eine **Sitzung** (30 Minuten) zurück. Die App schickt danach ein Sitzungs-Token mit – du musst den Code also nicht ständig neu eingeben. Läuft die Sitzung ab, meldest du dich einfach erneut an.

**Ohne TOTP-Geheimnis kein Admin.** Ist auf dem Server kein `ADMIN_TOTP_SECRET` gesetzt, ist der Admin-Zugang vollständig deaktiviert – niemand kommt rein.

Das Zugangs-Paket ( `.a3kpaket` ) ist dasselbe wie in der Nutzer-App. Du kannst dein vorhandenes verwenden oder in der Nutzer-App unter „Team verwalten“ ein eigenes erzeugen (z. B. *Admin-Laptop*).

### 3 Server vorbereiten (TOTP einrichten)

Einmalig richtest du das Admin-Geheimnis auf dem Server (NAS) ein.

- 1 **Geheimnis erzeugen.** Im `server/` -Ordner auf der NAS:

```
docker compose -f docker-compose.tailscale.yml run --rm api totp
```

Der Befehl gibt das Geheimnis, die `otpauth://` -URL und einen **QR-Code direkt im Terminal** aus (jeder Lauf erzeugt ein *neues, zufälliges* Geheimnis – die Zeichen unten sind nur ein Platzhalter):

```
ADMIN_TOTP_SECRET=XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

otpauth://totp/Antrag%203000:Admin?secret=...&issuer=Antrag%203000

(QR-Code zum Scannen)
```

**Geheim halten:** Dieses Geheimnis ist der zweite Admin-Faktor. Niemals in Dokumente, Chats oder ins Git übernehmen – nur in `server/.env` auf der NAS.

- 2 **Authenticator-App einrichten.** Den **QR-Code scannen** – oder den base32-Schlüssel manuell eintragen (Typ: zeitbasiert/TOTP).

- 3 **Geheimnis hinterlegen.** Die Zeile `ADMIN_TOTP_SECRET=...` in die Datei `server/.env` schreiben.

- 4 **Server neu bauen.** `docker compose -f docker-compose.tailscale.yml up -d --build`

**Geheimnis nicht verstreuen.** Die `otpauth://` -URL enthält das Geheimnis. Gib sie **nicht** in einen Online-QR-Dienst – nutze den Terminal-QR-Code oder die manuelle Eingabe.

**Uhrzeit.** TOTP braucht eine korrekte Uhr. Die Synology ist per NTP synchron; falls Codes nie akzeptiert werden, prüfe die Server-Uhrzeit.

#### Zwei Sicherheits-Schalter (optional, empfohlen)

In `server/.env` gibt es zwei Schalter, die das Team nach der Einrichtung zusätzlich absichern:

- **Geräte-Aufnahme einfrieren:** Sobald alle Team-Geräte einmal verbunden waren, `AUTO_ENROLL=0` setzen. Dann kommen **keine neuen Geräte** mehr von selbst dazu (auch nicht mit gültigem Zertifikat). Ein gesperrtes Gerät kann sich danach nicht neu anmelden.
- **Proxy festlegen:** `CERT_HEADER_MODE` ist in den Compose-Dateien bereits passend gesetzt ( `caddy` bzw. `cloudflare` ) – nur ändern, wenn du den Aufbau wechselst.

Nach Änderungen den Server neu starten: `docker compose -f docker-compose.tailscale.yml up -d --build .`

## 4 Admin-App installieren & starten

Die Admin-App liegt im Projekt unter `admin-app/`. Voraussetzung ist dieselbe Toolchain wie bei der Nutzer-App (Rust + Node).

### 1 Abhängigkeiten holen (einmalig):

```
cd admin-app  
npm install
```

### 2 Starten – zum Arbeiten/Testen:

```
npm run tauri dev
```

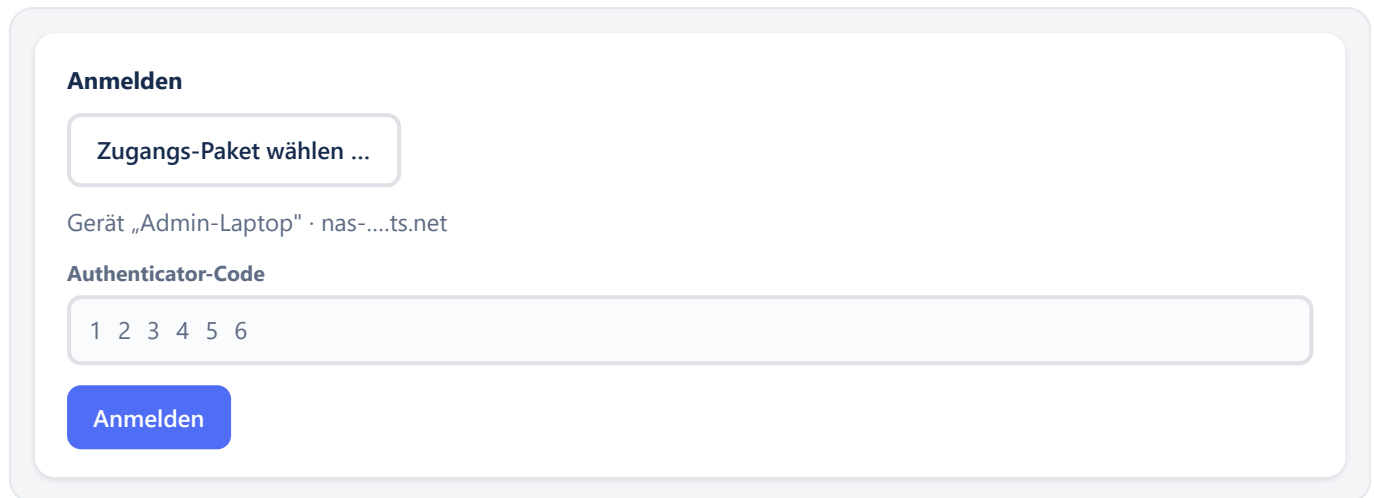
### 3 Fertige App bauen (optional, erzeugt eine Installations-Datei):

```
npm run tauri build
```

Die Admin-App ist bewusst schlank: eine statische Oberfläche und ein kleines Rust-Backend, das denselben gesicherten mTLS-Client wie die Nutzer-App nutzt.

## 5 Anmelden

Zugangs-Paket wählen, Code eingeben – fertig.



**Anmelden**

Zugangs-Paket wählen ...

Gerät „Admin-Laptop“ · nas-....ts.net

**Authenticator-Code**

1 2 3 4 5 6

Anmelden

**Abb. 5.1** – Paket wählen, 6-stelligen Code eingeben, anmelden.

- 1 Zugangs-Paket wählen** ( `.a3kpaket` ). Die App liest daraus Gerätenamen und Team-Adresse.
- 2 Code eingeben** – die aktuellen 6 Ziffern aus der Authenticator-App.
- 3 Anmelden.** Der Server prüft den Code und öffnet eine 30-Minuten-Sitzung. Danach erscheinen die vier Bereiche.

„**Code falsch oder abgelaufen**“? Meist ist der Code schon gewechselt (neu eingeben) – oder das Geheimnis steht nicht in der `server/.env` bzw. die Server-Uhr weicht ab.

## 6 Vorschläge kuratieren

Der Sammler schlägt neue oder geänderte Förderungen vor – verteilt wird erst, was du **freigibst**.

| Förderung               | Art      | Quelle · Stand              | Aktion    |           |
|-------------------------|----------|-----------------------------|-----------|-----------|
| Beispiel-Kulturstiftung | neu      | rohquelle.json · 22.06.2026 | freigeben | verwerfen |
| Musikfonds e. V.        | geändert | rohquelle.json · 22.06.2026 | freigeben | verwerfen |

Abb. 6.1 – Tab „Vorschläge“: je Eintrag freigeben oder verwerfen.

- **freigeben** übernimmt die Förderung sofort in den verteilten Katalog (neuer Eintrag oder Ersetzen des gleichen) und setzt das Katalog-Datum auf heute.
- **verwerfen** lehnt den Vorschlag ab. Findet der Sammler denselben Eintrag später *unverändert*, schlägt er ihn nicht erneut vor.

So funktioniert das **Hybrid-Prinzip**: Der Sammler arbeitet automatisch, aber **ein Mensch entscheidet**, was beim Team ankommt. Wie der Sammler Vorschläge erzeugt, steht in Kapitel 10.

### Ansehen, bevor du freigibst (Diff)

Mit **ansehen** öffnest du alle Felder eines Vorschlags. Bei einem **geänderten** Vorschlag sind die Felder, die sich vom aktuellen Katalog unterscheiden, **gelb** hervorgehoben (alter Wert durchgestrichen → neuer Wert) – so erkennst du auf einen Blick, was sich ändert, und gibst nicht „blind“ frei. Freigeben und verwerfen geht auch direkt aus diesem Fenster.

Hat der Vorschlag eine Webseite, kannst du sie mit **Website öffnen** sofort im Browser prüfen.



## 7 Meldungen bearbeiten

Nutzer:innen melden falsche oder veraltete Förderungen – hier arbeitest du sie ab.

| Förderung        | Art               | Anmerkung              | Status | Aktion   |           |
|------------------|-------------------|------------------------|--------|----------|-----------|
| Musikfonds e. V. | veraltete Angaben | „Frist ist abgelaufen“ | offen  | erledigt | verwerfen |

Abb. 7.1 – Tab „Meldungen“: Status auf erledigt/verworfen setzen.

- **erledigt** – die Meldung ist bearbeitet (z. B. der Katalog wurde korrigiert).
- **verwerfen** – die Meldung trifft nicht zu.
- **öffnen** – eine erledigte/verworfenen wieder auf „offen“ setzen.

Eine Meldung enthält nur Förderungs-Name, Art und die optionale Anmerkung der meldenden Person – keine sensiblen Daten.

### Detailansicht einer Meldung

Mit **ansehen** öffnest du eine Meldung. Oben steht **gelb markiert, was gemeldet wurde** (Art und Anmerkung), darunter der **aktuelle Katalog-Eintrag** der Förderung zum direkten Abgleich. Aus dem Fenster heraus kannst du

- mit **Website öffnen** die Förder-Webseite prüfen,
- mit **Förderung bearbeiten** die Angaben gleich korrigieren (siehe Kapitel 9), und
- die Meldung auf **erledigt** oder **verworfen** setzen.

## 8 Geteilte Förderer

Selbst recherchierte Förderer, die Team-Mitglieder geteilt haben – mit ihren **öffentlichen** Feldern.

Im Tab **Geteilte Förderer** siehst du Name, Geber, Land sowie Gerät und Stand. Passt ein Eintrag nicht (Dublette, unpassend, fehlerhaft), entfernst du ihn mit [entfernen](#).

Die **freie Beschreibung** eines geteilten Förderers bleibt beim Ersteller lokal – sie wird gar nicht erst übertragen. Du siehst also nur die öffentlichen Programm-Infos.

### In den Katalog übernehmen

Einen guten geteilten Förderer übernimmst du mit [→ in Katalog](#) dauerhaft: Es öffnet sich das Bearbeiten-Formular (Kapitel 9), **vorausgefüllt** mit den geteilten Angaben und einer automatisch erzeugten ID. Du ergänzt fehlende Felder und speicherst – der Eintrag landet sofort im verteilten Katalog. Eine Datei von Hand zu bearbeiten ist dafür nicht mehr nötig.

## 9 Katalog hochladen

Der zentrale Förder-Katalog wird auf den Server geladen und von dort an alle Team-Geräte verteilt.

- 1 Katalog-Datei wählen.** Im Tab „Katalog hochladen“ auf „Katalog-Datei wählen ...“ klicken und die `foederungen.json` aussuchen. Die App zeigt Anzahl der Förderungen und den Stand.
- 2 „Ergänzen“ wählen (empfohlen).** Fügt die Förderungen aus der Datei zum *bestehenden* Katalog hinzu: gleiche `id` wird aktualisiert, neue kommen dazu, alles andere bleibt erhalten. Vor dem Hochladen erscheint eine Vorschau („X neu, Y aktualisiert“). Ideal für neu recherchierte Förderungen – z. B. eine Scraper-Datei mit nur den neuen Einträgen.
- 3 Oder „Ersetzen (alles)“.** Wirft den *gesamten* bisherigen Katalog weg und nimmt nur die Datei. Eine Sicherheitsabfrage warnt ausdrücklich, wenn die Datei weniger Förderungen enthält als der Server. Nur nutzen, wenn die Datei wirklich den vollständigen Katalog enthält.

**Wichtig:** „Ersetzen“ überschreibt alles und kann nicht rückgängig gemacht werden. Wenn du nur neue Förderungen hinzufügen willst, nimm **„Ergänzen“** – sonst gehen die bisherigen verloren.

Der Katalog liegt nach dem Upload in der Server-Datenbank. Beim Abruf liefert der Server diese Fassung; ohne Upload greift die mitgelieferte Datei. Die Nutzer-App übernimmt Updates automatisch und zeigt, was sich geändert hat.

**Abgelehnt?** „Der Katalog ist ungültig“ heißt: Schema-Version fehlt/passt nicht, oder eine Förderung hat keine `id`/keinen Namen. „Zu groß“ greift erst bei sehr großen Dateien (über 2 MB).

### Einzelne Förderungen bearbeiten (Tab „Förderungen“)

Statt jedes Mal die ganze Datei zu tauschen, kannst du im Tab **Förderungen** einen einzelnen Eintrag ändern.

Die Liste zeigt alle Förderungen mit Suchfeld; **bearbeiten** öffnet ein Formular, in dem **alle Angaben**

**Bedienelemente sind** (kein JSON von Hand):

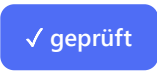
- Textfelder für Name, Geber, Webseite, Förderhöhe, Beschreibung;
- Auswahllisten für Land, Währung und **Zeitpunkt** – inkl. der Möglichkeit „wiederkehrend (z. B. halbjährlich)“ für Förderer mit regelmäßig wiederkehrenden Fristen;
- Mehrfachauswahl per Häkchen für Trägerschaft, Sparten, Projektarten, erlaubte Länder und – ausklappbar – Regionen;
- einen **Fristen-Editor**: je Frist ein Datum und ein optionaler Hinweis. Das Datum darf **mit Jahr** (15.09.2026) oder **ohne Jahr** (15.09.) eingegeben werden – ohne Jahr bedeutet „jährlich wiederkehrend“. Dazu ein **allgemeiner Frist-Hinweis** (z. B. „für das erste Halbjahr“ je Frist oder „mind. 3 Monate vor Projektstart“, auch bei laufender Einreichung);
- Zeilen-Listen für Städte, „unverträglich mit“ und den Checklisten-Vorschlag;
- eine Option **„Einreichung nur über Online-Formular“** mit Adressfeld – ist sie gesetzt, zeigt die Nutzer-App statt „per Mail senden“ einen Verweis auf dieses Online-Formular.

**Speichern** übernimmt die Änderung sofort in den verteilten Katalog und setzt den Stand auf heute. Mit

 prüfst du die Förder-Webseite. Felder, die das Formular nicht kennt, bleiben dabei erhalten.

## Regelmäßige Prüfung (Tab „Prüfung“)

Jede Förderung sollte **alle 3 Monate** einmal von Hand geprüft werden (stimmen Angaben, Fristen und Webseite noch?). Der Tab **Prüfung** hilft dabei:

- Eine Übersicht zeigt oben, wie viele Förderungen **überfällig**, **bald fällig** oder **aktuell** sind.
- Die Liste ist nach Dringlichkeit sortiert (Überfällige zuerst); mit „nur fällige zeigen“ blendest du die aktuellen aus.
- Je Zeile:  öffnet die Webseite,  öffnet das Formular,  setzt das Prüfdatum auf heute.

„✓ geprüft“ und auch ein **Bearbeiten** setzen das Prüfdatum auf heute. Das reine Abhaken löst **kein** Update bei den Nutzer:innen aus (es ändert nur eine interne Prüf-Notiz, nicht den Katalog-Stand) – ein echtes Bearbeiten dagegen schon.

## 10 Der Sammler (Hybrid-Update)

Der Sammler vergleicht eine Rohquelle mit dem aktuellen Katalog und legt daraus die Vorschläge an, die du in Kapitel 6 freigibst.

### Die Rohquelle: woher die Kandidaten kommen

Die **Rohquelle** ist der Feed der recherchierten Förderer. Sie entsteht aus dem gepflegten Katalog im Projekt und wird mit einem Befehl erzeugt:

```
node import/rohquelle-bauen.mjs
```

Das schreibt `server/katalog/rohquelle.json` (mit Prüfung aller Regions-/Stadt-Codes). Diese Datei legst du auf der NAS in den gleichen `katalog/` -Ordner wie `foerderungen.json` – im Container ist sie dann unter `/srv/rohquelle.json` erreichbar.

### Einen Sammler-Lauf starten

Auf der NAS im `server/` -Ordner:

```
docker compose -f docker-compose.tailscale.yml run --rm api sammeln /srv/rohquelle.json
```

Der Sammler meldet z. B. „2 neu, 1 geändert“. Diese Vorschläge erscheinen anschließend im Tab **Vorschläge**. Der Lauf nutzt einen Wegwerf-Container und stört den laufenden Server nicht (gemeinsame Datenbank dank WAL-Modus).

### Wie verglichen wird

- **neu** – die id kommt im Katalog noch nicht vor.
- **geändert** – die id gibt es, aber der Inhalt unterscheidet sich.
- **unverändert** – wird übersprungen (kein Vorschlag).

Der Lauf ist **wiederholbar**: Gleiche, bereits gesehene Kandidaten erzeugen keinen neuen Vorschlag; ein einmal verworfener kommt erst wieder, wenn sich sein Inhalt tatsächlich ändert.

**Wöchentlich automatisch.** Statt den Befehl von Hand zu tippen, gibt es das fertige Skript `server/sammler-cron.sh`. Trag es im *Synology-Aufgabenplaner* als wöchentliche Aufgabe ein (Systemsteuerung → Aufgabenplaner → Erstellen → Geplante Aufgabe → Benutzerdefiniertes Skript), Befehl `sh /pfad/zu/server/sammler-cron.sh`. Verteilt wird trotzdem nichts ohne deine Freigabe – der Sammler legt nur Vorschläge an.

Eine Beispiel-Rohquelle zum Ausprobieren liegt unter `server/katalog/rohquelle-beispiel.json`; die echte erzeugst du wie oben mit `node import/rohquelle-bauen.mjs`.

## 11 App-Updates veröffentlichen

Die App aktualisiert sich selbst – aber nur mit Paketen, die mit **deinem privaten Signatur-Schlüssel** unterschrieben sind. Der öffentliche Gegenpart steckt fest in der App; eine kompromittierte NAS kann so keinen Schadcode verteilen.

### Einmalig: Signatur-Schlüssel

- Schlüsselpaar erzeugen:

```
npm run tauri signer generate -- -w "$HOME\.tauri\antrag3000.key"
```

- **Privaten** Schlüssel ( `antrag3000.key` ) und Passwort sicher verwahren – nie ins Git, nie auf die NAS.
- Den **öffentlichen** Schlüssel ( `...key.pub` ) in `src-tauri/tauri.conf.json` unter `plugins.updater.pubkey` eintragen.

### Pro Release

1. Versionsnummer erhöhen (gleich in `tauri.conf.json` , `Cargo.toml` , `package.json` ).
2. Signiert bauen (PowerShell):

```
$env:TAURI_SIGNING_PRIVATE_KEY = Get-Content "$HOME\.tauri\antrag3000.key" -Raw

$env:TAURI_SIGNING_PRIVATE_KEY_PASSWORD = "DEIN-PASSWORT"

npm run tauri build
```

3. Manifest erzeugen:

```
node release/latest-json-bauen.mjs "Was ist neu"
```

4. Auf die NAS in den Ordner `updates/` hochladen: den `...-setup.exe` -Installer **und** `latest.json` (die `.sig` nicht – ihr Inhalt steht im Manifest). Wirklich überschreiben.

Beim nächsten Start (oder per Knopf „**↑** Update“) sehen alle Nutzer:innen die neue Version, bestätigen, und die App aktualisiert sich und startet neu.

**Schlüssel verloren?** Dann kannst du keine Updates mehr signieren. Du müsstest ein neues Paar erzeugen, den neuen öffentlichen Schlüssel einbauen und diese eine neue Version **einmalig von Hand** verteilen. Bewahre privaten Schlüssel und Passwort darum sicher auf. Ausführlich: `docs/release-bauen.md` .

## 12 Tipps & häufige Fragen

---

### **Brauche ich ein besonderes Admin-Zertifikat?**

Nein. Es genügt ein gültiges Team-Zugangs-Paket; der zweite Faktor ist der TOTP-Code. Du kannst für die Admin-Arbeit aber ein eigenes Paket (z. B. *Admin-Laptop*) in der Nutzer-App erzeugen.

### **Ich habe mein Authenticator-Gerät verloren.**

Erzeuge auf dem Server mit `server totp` ein neues Geheimnis, trage es in `server/.env` ein und baue den Server neu. Das alte Geheimnis ist damit ungültig.

### **„Sitzung abgelaufen – bitte neu anmelden.“**

Die 30-Minuten-Sitzung ist abgelaufen. Einfach erneut mit aktuellem Code anmelden.

### **Verändert die Admin-App sensible Nutzerdaten?**

Nein. Sie arbeitet ausschließlich auf der öffentlichen Förder-Ebene (Katalog, Meldungen, geteilte Förderer). Tresor-Daten erreicht sie nie.

### **Wie kommen meine Freigaben bei den Nutzer:innen an?**

Freigegebene Vorschläge und hochgeladene Kataloge liefert der Server beim nächsten Abruf aus. Die Nutzer-App übernimmt Updates automatisch und zeigt die Änderungen an.

— Ende des Admin-Handbuchs · Antrag 3000, Version 0.2.0 —